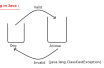
What is upcasting and downcasting in Java :

upcasting :



Upcasting :

* In java, It is possible to assign to **sub class object to super class reference variable is called upcasting.**

* The upcasting concept comes under **dynamic polymorphism.**

Example :

```
class Animal
{
    public void run()
    {
        SO.println("Generic Animal is roaming");
    }
}
class Dog extends Animal
{
    public void run() //Override method
    {
        SO.println("Dog Animal is roaming");
    }
}
```

Animal animal = new Dog(); //Upcasting

Different cases for calling static & non static methods using upcasting:

Different Cases	Non static Method	Static Method
Animal a = new Animal(); a.run();	Java compiler will **start searching** run() method in the Animal class and JVM will **start executing** from Animal class	Java compiler will **start searching** run() method in the Animal class and JVM will **start executing** from Animal class
Dog dog = new Dog(); Dog.run();	Java compiler will **start searching** run() method in the Dog class and JVM will **start executing** from Dog class	Java compiler will **start searching** run() method in the Dog class and JVM will **start executing** from Dog class
Animal a = new Dog(); a.run();	Java compiler will **start searching** run() method from Animal class and JVM will **start executing** from Dog class.	Java compiler will **start searching** run() method from Animal class and JVM will **start executing** from Animal class. Static methods are hidden not overridden

//OverridingDemo1.java

```
package com.ravi.overriding;

class Animal
{
    public void run()
    {
        SO.println("Generic Animal is roaming");
    }
}
class Dog extends Animal
{
    public void run()
    {
        SO.println("Dog animal is roaming");
    }
}

class Puppy extends Dog
{
}
class Lion extends Animal
{
    public void run()
    {
        SO.println("Lion animal is roaming");
    }
}
public class OverridingDemo1
{
    public static void main(String[] args)
    {
        Animal a1 = new Puppy();
        a1.run();

        Animal a2 = new Lion();
        a2.run();
    }
}
```

@Override Annotation :

* In Java, Annotation concept is available from JDK 1.5V.

* Annotations are always starts with @ symbol.

* @Override annotation is **optional** but it is strongly recommended to write @Override annotation just before the method declaration. It helps the compiler to understand that we are overriding method that this particular method is available in the super class as well as it is an overridden method.

* If the method is not an overridden method then code will not compile.

```
package com.ravi.overriding;

class Bird
{
    public void fly()
    {
        SO.println("Generic Bird is flying");
    }
}

class Parrot extends Bird
{
    @Override
    public void fly()
    {
        SO.println("Parrot Bird is flying");
    }
}

class Peacock extends Bird
{
    @Override
    public void fly()
    {
        SO.println("Peacock Bird is flying");
    }
}
public class OverridingDemo2 {
    public static void main(String[] args)
    {
        Bird bird = null;

        bird = new Parrot();  bird.fly();  //Dynamic Method Dispatch
        bird = new Peacock();  bird.fly();  //Dynamic Method Dispatch
    }
}
```

Can we override private method :

No, We can't override private method of super class because private methods are not visible (not available) to the sub class in the inheritance.

No, we can't use @Override annotation on overridden method of sub class if super class method is declared with private access modifier because it is not overridden method, actually it is re-declared by sub class developer.

OverridingDemo2.java

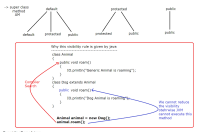
```
package com.ravi.overriding

class Super
{
    private void m1()
    {
        System.out.println("M1 method in super class");
    }
}
class Sub extends Super
{
    public void m1() //Re-Declaration
    {
        System.out.println("M1 method in sub class");
    }
}
public class OverridingDemo2
{
    public static void main(String[] args)
    {
        //Super s = new Sub();
        //s.m1();//error:m1 method is not visible

        new Sub().m1();
    }
}
```

Rule of access modifier while overriding a method :

* At the time of overriding the method, we **cannot reduce the visibility** of the method.

* Visibility means, the **access modifier of sub class method must be greater OR should be equal in comparison to the access modifier of super class method.**

* In terms of visibility, **public > protected > default > private**
(Note : private methods are not overriden)



OverridingDemo4.java

```
package com.ravi.overriding;

class SuperClass
{
    public void show()
    {
        SO.println("Show method of super class");
    }
}
class SubClass extends SuperClass
{
    @Override
    protected void show() //error
    {
        SO.println("Show method of sub class");
    }
}

public class OverridingDemo4 {
    public static void main(String[] args)
    {
        SuperClass superClass = new SubClass();
        superClass.show();
    }
}
```

Variable hiding concept with Upcasting :

```
class Super
{
    int x = 100;

class Sub extends Super
{
    int x = 200; //variable hiding
}
```

Only non static methods are overridden in java but not the variables(variables are not overridden in java) because behavior of an object (methods) will change but not the properties(variables).

Note : In upcasting, non static fields will be always executed based on the current reference whereas methods are executed based on the object.

```
package com.ravi.overriding;

class RBI
{
    protected String IFSCCode = "RBI0000001";

    public String loan();
    {
        return "Bank should provide loan";
    }
}
class SBI extends RBI
{
    protected String IFSCCode = "SBI0000000";//SBI

    @Override
    public String loan();
    {
        return "SBI provides the loan @ 9.5% ROI";
    }
}

public class OverridingDemo5
{
    public static void main(String[] args)
    {
        RBI rbi = new SBI();
        SO.println(rbi.IFSCCode+" : "+rbi.loan());
    }
}
```

What is downcasting in Java ?

Example :

```
class Animal
{
    public void run()
    {
        SO.println("Generic Animal is roaming");
    }
}
class Dog extends Animal
{
    public void run()
    {
        SO.println("Dog Animal is roaming");
    }
}

Dog d = (Dog) new Animal();  //It is not downcasting [This statement is not possible in Java]
                             //[Java.lang.ClassCastException: Animal cannot be cast to Dog
```

* Downcasting is mainly used to call **specific method.**

* It is not possible without up-casting. (If there is no upcasting then there is no down casting)

* It is a technique through which we can assign **sub class object to sub-class reference variable by using super class reference.**